Infinite Generators

A generator with `while True: yield` never ends – you pull exactly as many values as you need.

The pattern

```
def naturals(start=1):
    n = start
    while True:           # never ends...
        yield n
        # ...but costs nothing until asked
        n += 1

nat = naturals()
print(next(nat), next(nat), next(nat))
```

OUTPUT

```
1 2 3
```

Take only what you need

```
from itertools import islice, takewhile

print(list(islice(naturals(), 5)))
print(list(islice(naturals(10), 2, 6)))
print(list(takewhile(lambda n: n < 5, naturals())))
```

OUTPUT

```
[1, 2, 3, 4, 5]
[12, 13, 14, 15]
[1, 2, 3, 4]
```

Tool	Does
`islice(it, stop)`	first `stop` items
`islice(it, start, stop)`	a slice of the stream
`takewhile(pred, it)`	items until `pred` first fails
`zip(a, endless)`	stops at the shortest input
`for … break`	manual stop condition

Classic infinite generators

```
def fib():
    a, b = 0, 1
    while True:
        yield a
        a, b = b, a + b

print(list(islice(fib(), 10)))

def primes():
    found, n = [], 2
    while True:
        if all(n % p for p in found):
            found.append(n)
            yield n
        n += 1

print(list(islice(primes(), 8)))
```

OUTPUT

```
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
[2, 3, 5, 7, 11, 13, 17, 19]
```

Ready-made infinite iterators

Function	Produces
`count(10, 5)`	10, 15, 20, …
`cycle("AB")`	A, B, A, B, …
`repeat(7)`	7, 7, 7, … (`repeat(7, 3)` stops after 3)

```
from itertools import count, cycle, repeat
print(list(islice(count(10, 5), 4)))
print(list(islice(cycle("AB"), 5)))
print(list(repeat(7, 3)))
```

OUTPUT

```
[10, 15, 20, 25]
['A', 'B', 'A', 'B', 'A']
[7, 7, 7]
```

Round-robin and running totals

```
from itertools import accumulate
servers = cycle(["a", "b", "c"])
print([next(servers) for _ in range(5)])
print(list(islice(accumulate(count(1)), 5)))
```

OUTPUT

```
['a', 'b', 'c', 'a', 'b']
[1, 3, 6, 10, 15]
```

Infinite generator expressions & zip

```
ids = (f"ID-{n:03d}" for n in count(1))
print(next(ids), next(ids))
for ch, n in zip("xyz", naturals()):
    print(ch, n)
```

OUTPUT

```
ID-001 ID-002
x 1
y 2
z 3
```

PITFALL `list(naturals())`, `sum(naturals())` or a `for` loop without `break` never finishes and can eat all your memory. Always bound the stream with `islice`, `takewhile`, `zip` or `break`.

TIP Great for ID generators, event streams, round-robin scheduling and retry back-off timers.